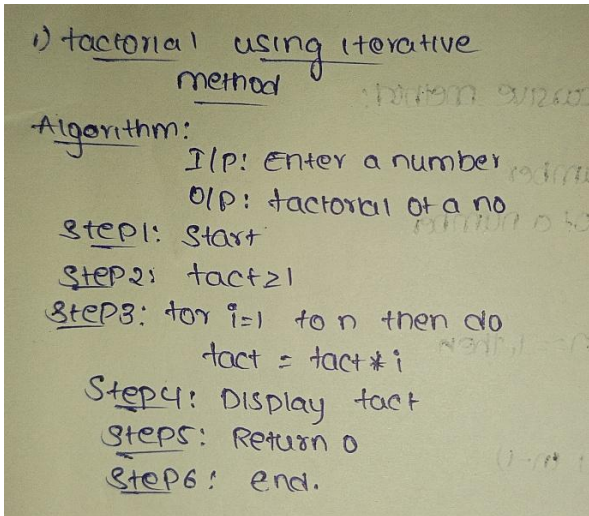


Practical-04

AIM: Implementation and Time analysis of factorial program using iterative and recursive method

1) ITERATIVE METHOD

ALGORITHM:



1) factorial using iterative method

Algorithm:

I/p: Enter a number
O/p: factorial of a no

Step 1: Start

Step 2: fact = 1

Step 3: for i = 1 to n then do
fact = fact * i

Step 4: Display fact

Step 5: Return 0

Step 6: end.

PROGRAM:

```
#include <iostream>

using namespace std;

int main(){

    int n;

    int fact = 1;

    cout << "enter a number:";

    cin >> n;

    for(int i = 1; i <= n; i++){

        fact = fact * i;

    }
```

Name: S. Jaswanth siva kumar

Enrolment number: 92460118154

```
cout << "Factorial of " << n << " = " << fact;  
return 0;  
}
```

OUTPUT:

```
Output  
enter a number:5  
Factorial of 5 = 120  
=== Code Execution Successful ===
```

TIME COMPLEXITY:

Time complexity:

i) Best case:
loop runs 1 to n
fact = 1
for i = 1 to n
fact = fact * i
 $T(n) = (1 + n + n) = 2n + 1$
 $T(n) = O(n)$

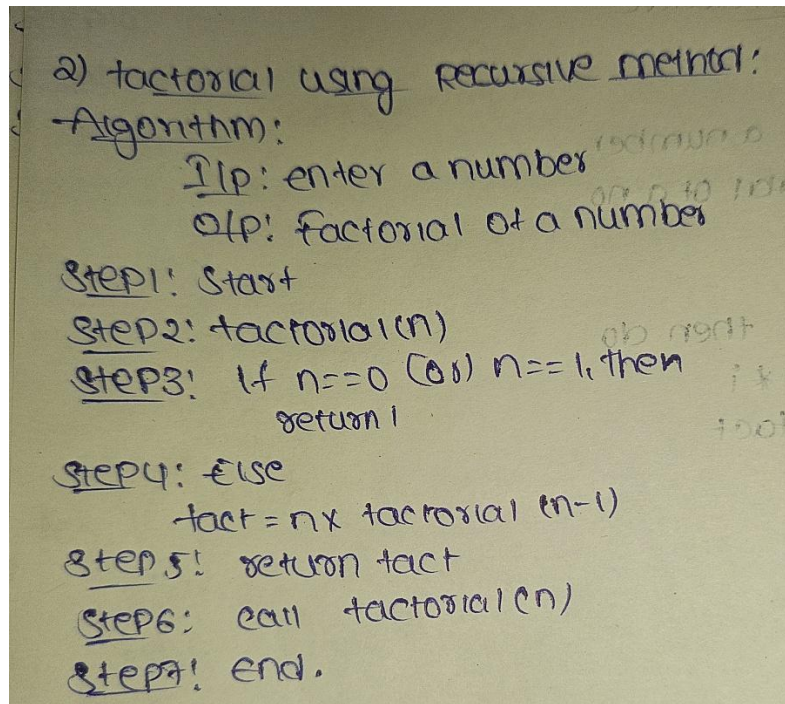
ii) Average case
loop always executes 'n' times
i = 1 → 1
i = 2 → 1
i = 3 → 1
i = n → 1
no. of iteration = n
 $T(n) = O(n)$

iii) Worst case:
It is also loop executed 'n' times
1, 2, 3, 4, ..., n
 $T(n) = (1 + n + n) = 2n + 1$
 $T(n) = O(n)$

Best = Average = Worst = $O(n)$

2) RECURSIVE METHOD:

ALGORITHM:



PROGRAM:

```
#include <iostream>

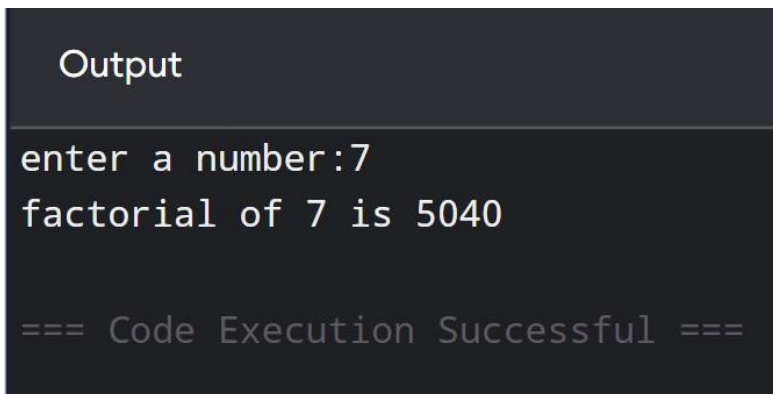
using namespace std;

class factorial{
public:
    int factr(int n){
        if(n == 0 || n == 1){

            return 1;
        }
    }
}
```

```
int result = n*factr(n-1);  
return result;  
}  
}  
};  
  
int main(){  
    int n,result;  
    factorial obj;  
    cout << "enter a number:";  
    cin>>n;  
    result = obj.factr(n);  
    cout << "factorial of " << n << " is " << result;  
    return 0;  
}
```

OUTPUT:



Output

```
enter a number:7  
factorial of 7 is 5040  
  
=== Code Execution Successful ===
```


TIME COMPLEXITY:

Time complexity:

factorial(n) $\rightarrow$ n
if (n == 0 || n == 1) $\rightarrow$ n
 n * factorial(n-1) $\rightarrow$ (n-1)
 return fact $\rightarrow$ n
T(n) = n + n + (n-1) + n
 = 4n+1
T(n) = O(n)

It is worst case Time complexity = O(n)

i) Best case:
When n=0 or n=1
factorial(1) $\rightarrow$ 1
T(n) = O(1)

ii) Average case:
no. of calls = n
ex: n=5 $\Rightarrow$ 5 $\rightarrow$ 4 $\rightarrow$ 3 $\rightarrow$ 2 $\rightarrow$ 1
T(n) = O(n)

iii) Worst case:
large no. of n continues until it reaches 1
ex: n $\rightarrow$ n-1 $\rightarrow$ n-2 $\rightarrow$... $\rightarrow$ 2 $\rightarrow$ 1
T(n) = O(n)

CONCLUSION: Both iterative and recursive methods are used to find factorial. Both take O(n) time, but the iterative method uses less memory and is simpler to implement.